AI-Driven 소프트웨어 개발 실습 | WS0

Problem Description 기반 AI 코드 생성 관찰 보고서

D1 vs D2 — 화면명세 텍스트 추가가 AI 코드 생성에 미치는 영향

팀명 : **Meet2Action**

시스템명 : **Meet2Action 스마트 워크플로우**

작성일 : **2026. 03. 30**

이번 회차 학습 목표 및 실험 설계

학습 목표

UML 모델링 없이 Problem Description만으로 AI 코드를 생성하여 "설계 명세 없이 AI에게 맡기면 어떤 코드가 나오는가"를 관찰하는 Baseline을 확보한다.
이후 회차에서 모델링이 추가될 때마다 이 결과와 비교한다.

실험 설계

D1 기본 프롬프트

PD (Problem Description)

+ Cancel 상태전이 명시

+ 제약조건 7개

→ 화면명세 없음

D2 화면명세 추가

D1의 모든 구성 포함

+ [화면 구성] 블록 추가

7개 화면 + 연관 엔티티 기술

→ 화면명세 텍스트 포함

비교 관찰 포인트

엔티티 수 변화

DB 테이블 구조 차이

추가 기능 생성 여부

AI 임의 결정 항목 차이

D1 기본 프롬프트

[역할 지정]
Java로 Meet2Action 시스템의 전체 기능을 구현하라.
외부 프레임워크 없이 순수 Java로 작성한다.
데이터는 PostgreSQL에 JDBC로 직접 연결하여 저장한다.
UI/화면 구현 없이 콘솔 출력 수준으로 구현한다.

[컨텍스트 - Problem Description]
Meet2Action은 팀 회의에서 발생하는 비정형 텍스트를 AI 분석 엔진을 통해 실행 가능한 Action Item으로 자동 변환하고, 팀 업무 진척 상태를 통합 관리하는 스마트 워크플로우 시스템이다.

시스템은 클라이언트 단말(사용자 브라우저/앱)과 중앙 백엔드 서버가
분리된 분산 구조(Client ↔ Server)로 구성된다.
서버는 AI 분석, 데이터 저장, 권한 제어의 핵심 로직을 처리한다.

사용자는 세 종류이다.
- Admin : 시스템 전반 관리, 검색, 권한 통제
- Regular User : 회의 입력, Action Item 승인 및 할당
- Participant : 본인 Task 확인 및 진척도 업데이트

[생성 지시]
아래 기능 전체를 구현하라.

1. 회의 정보 및 텍스트 입력
2. AI 기반 Action Item 추출
3. Action Item 수정 및 승인
4. 업무 진척 상태 관리
5. Action Item 취소
6. Task 검색 및 대시보드

[상태 전이]
Action Item은 아래 순서로만 전이된다. 역방향 불가.
TEXT_INPUT_COMPLETED
→ ANALYZING
→ ACTION_ITEM_GENERATED
→ TASK_ASSIGNED
→ IN_PROGRESS
→ COMPLETED

취소 분기:
ACTION_ITEM_GENERATED → CANCELLED (승인 전, 누구나)
TASK_ASSIGNED → CANCELLED (Admin/Regular User만)
IN_PROGRESS → CANCELLED (Admin만)
COMPLETED → 취소 불가
CANCELLED → 추가 전이 없음

상태는 Java enum으로 정의하라.
허용되지 않은 전이 시 예외를 발생시키고 현재 상태를 유지하라.

[제약 조건]
- 회의 텍스트가 비어있으면 분석을 거부한다
- Participant의 승인 시도는 거부한다
- 미등록 사용자에게 Task 할당 시도는 거부한다
- 이미 할당된 Task 재할당 요청 시 확인 절차를 수행한다
- COMPLETED Task의 상태 변경은 거부한다
- CANCELLED Task의 재활성화는 거부한다
- 복수 사용자가 동일 Task에 동시 접근 시
서버 측 임계 영역(synchronized)에서 제어한다

D2 화면명세 추가

[화면 구성 - 참고]
아래 화면 구성을 참고하여 필요한 엔티티와 데이터 관계를 설계에 반영하라. 화면 구현은 하지 않아도 되며,
엔티티 및 데이터 구조 설계의 참고 자료로만 활용한다.
1. 대시보드
- 데이터: 전체 업무 현황(진행중/완료/대기 건수),
최근 회의 목록, 업무 현황 그래프
- 연관: Meeting, Task
2. 회의 관리
- 데이터: 회의 목록(제목/날짜/참여자), 날짜·참여자·키워드 필터
- 연관: Meeting, User
회의 상세
- 데이터: 회의 정보(제목/날짜), 회의 원문 텍스트,
참여자 목록, Action Item 목록
- 연관: Meeting, Topic, Task, User
3. 회의 기록 입력
- 데이터: 텍스트 입력, 문서 업로드
- 저장 대상: 회의 텍스트, 첨부파일
4. 업무 관리 (Action Item)
- 데이터: 업무명/담당자/상태/기한 목록,
상태·담당자 필터
- 상태값: 생성/진행중/완료/취소
- 연관: Task, User
업무 상세
- 데이터: 업무 정보, 상태 변경 버튼,
상태 변경 히스토리
- 연관: Task, TaskHistory

<생략>

7. 알림
- 데이터: 업무 할당 알림, 상태 변경 알림,
회의 일정 알림, 읽음/미읽음 표시
- 연관: Notification, Task, Meeting

Part A. 프롬프트 구성 비교 (Step 1)

두 프롬프트 공통 구성: [역할 지정] [컨텍스트 - PD] [생성 지시 6개 기능] [상태 전이] [제약 조건]

D1 기본 프롬프트

[역할 지정] Java + PostgreSQL + JDBC + 콘솔 출력

[컨텍스트] Problem Description 원문

[생성 지시] 6개 기능 전체

[상태 전이] 6단계 파이프라인 + CANCELLED 분기

[제약 조건] 7개 거부·검증 조건 + synchronized

*파일명: WS0_Meet2Action_pmt_**D1**.md*

+

D2 화면명세 추가 프롬프트

[역할 지정] D1과 동일

[컨텍스트] D1과 동일

[생성 지시] D1과 동일

[상태 전이] D1과 동일

[제약 조건] D1과 동일

[화면 구성] 7개 화면 + 연관 엔티티 명시 추가

*파일명: WS0_Meet2Action_pmt_**D2**.md*

Part B-1. 코드 구조 체크리스트

관찰 항목	D1	D2	비고	근거
정상 흐름 주요 기능 구현	✓ 있음	✓ 있음	양쪽 모두 6개 기능 완전 구현	동일
대안 흐름(예외 처리) 구현	✓ 있음	✓ 있음	권한·상태·입력 검증 예외 처리	동일
Cancel 처리 구현	✓ 있음	✓ 있음	CANCELLED 전이 + 재활성화 거부 구현	동일
Operator(Admin) 기능 구현	✓ 있음	✓ 있음	ADMIN 권한 제어 구현	동일
주요 기능별 메서드 분리	✓ 있음	✓ 있음	Service 레이어 메서드 분리	동일
알림(Notification) 기능	✗ 없음	✓ 있음	알림 화면 명세 → NotificationService 생성	D2 추가
Task 이력(History) 기능	✗ 없음	✓ 있음	업무 상세 히스토리 → task_history 테이블 생성	D2 추가
대시보드 집계	✓ 있음	✓ 있음	D2는 DashboardStats 클래스 별도 생성	동일

✓ 있음 = 구현됨 ✗ 없음 = 미구현 D2 추가 = 화면명세 하네싱으로 인해 D2 에서만 생성됨

Part B-1. 엔티티 & DB 테이블 구조 비교

D1 — 5개 테이블 (AI 자체 추론)

users
user_id, username, email, role VARCHAR(20)

meetings
meeting_id, title, date, text, status VARCHAR(50)

meeting_participants
meeting_id + user_id (N:M 연결)

action_items
action_item_id, meeting_id, topic, decision, status

tasks
task_id, action_item_id, description, assignee_id, status

D2 — 7개 테이블 (화면명세 반영)

users
role CHECK + created_at 추가

meetings
status CHECK + created_at, updated_at 추가

meeting_participants
N:M 연결 (동일)

action_items
approved_by FK + approved_at 추가

tasks
task_name + due_date ★ + GIN 인덱스

task_history ★
from/to_status, changed_by, memo → 이력 관리

notifications ★
type CHECK, is_read, related_entity → 알림 관리

★ 화면명세 [화면 구성] 블록으로 인해 추가 생성된 테이블

Part B-2. AI가 임의로 결정한 부분

항목	AI의 결정 내용	D1 vs D2	PD 근거
AI 분석 엔진 시뮬레이션 방식	'주제:'/'결정:'/'할일:' 접두어 키워드 기반 파싱으로 Topic/Task/Decision 분류	D1·D2 동일 (화면명세 영향 없음)	없음
테스트 시나리오 구조	D1: 5개 Scenario 클래스 + TestResult 분리 D2: 단일 Main STEP 1~10 순차 실행	D1 ≠ D2 구조 다름	없음
예외 클래스 명명 방식	D1: PermissionDeniedException / ValidationException D2: UnauthorizedOperationException / InvalidInputException	D1 ≠ D2 명칭 다름	없음
Task 필드 구성	D1: description 단일 필드 D2: task_name + description 분리 + due_date 추가	D2에서 확장 (기한 필드 추가)	**D2만 있음 (화면명세 '기한' 항목)**
알림 타입 ENUM 정의	D1: 없음 D2: TASK_ASSIGNED / STATUS_CHANGED / MEETING_SCHEDULED 등 5종	D2에서 추가 생성	**D2만 있음 (알림 화면 명세)**

Part A Step 2. 코드 생성 실행 결과 (Docker 빌드 검증)

D1 빌드 & 실행 결과

✓ 빌드 성공 (26.2s, 60건 전체 PASS)

Scenario 1 상태 전이 검증	13/13 PASS
Scenario 2 취소 권한 검증	12/12 PASS
Scenario 3 Task 전이 & 권한	11/11 PASS
Scenario 4 AI 분석 엔진	10/10 PASS
Scenario 5 제약 조건 검증	14/14 PASS

총 60건 | PASS 60 | FAIL 0

D2 빌드 & 실행 결과

✓ 빌드 성공 (31.5s, 10 STEP 모두 완료)

STEP 1~2	사용자 등록 & 회의 생성	✓
STEP 3	AI 분석 → Action Item 추출	✓
STEP 4	승인 & 할당 + 권한 검증	✓
STEP 5	Task 상태 전이 + 이력 조회	✓
STEP 6	Action Item 취소 시나리오	✓
STEP 7	검색 (키워드/담당자/날짜)	✓
STEP 8	대시보드 집계	✓
STEP 9	알림 조회 & 읽음 처리	✓
STEP 10	빈 텍스트 분석 거부	✓

10 STEP 완료 | 예외 시나리오 포함 전체 검증

Part C. 코드 관찰 후 느낀 점

Q1. AI가 PD 근거 없이 스스로 결정한 항목 중 가장 의외였던 것은 무엇입니까?

D1과 D2의 테스트 시나리오 구조가 완전히 달랐다는 점이 가장 의외였다.
D1은 5개의 Scenario 클래스와 TestResult 객체로 구성된 체계적인 단위 테스트 구조를 생성했고,
D2는 단일 Main 클래스에서 STEP 1~10으로 순차 실행하는 통합 테스트 방식을 선택했다.

두 프롬프트의 테스트 구조 지시는 동일했음에도 불구하고 AI가 완전히 다른 방식으로 추론한 것은,
화면명세 텍스트가 추가되면서 AI가 시스템을 '단계별 워크플로우 시스템'으로 더 강하게 인식했기 때문으로 보인다.

Q2. AI가 코드를 생성할 때 가장 부족했던 정보는 무엇이라고 생각합니까?

화면명세(Functional Specification)와 엔티티 간의 명시적 연관 관계 정의가 가장 부족했다.

D2에서 [화면 구성] 블록에 '연관 엔티티'를 텍스트로 명시하자 task_history와 notifications 테이블이
자동으로 생성되었다. 이는 화면 단위의 데이터 요구사항이 엔티티 설계에 직접적인 영향을 준다는 것을 의미한다.

D1에서 이 두 테이블이 없었던 이유는 PD만으로는 '이력을 어디에 어떻게 저장할지'에 대한
힌트가 전혀 없었기 때문이다. 다음 회차(WS1 Use Case Modeling)에서 이 부분이 어떻게 보완되는지 비교할 예정.

결론 및 시사점

화면명세 텍스트 추가는 엔티티 설계에 직접 영향을 준다

01

Cancel 처리 성공

상태 전이 명세를 프롬프트에 명시하자 D1·D2 모두 CANCELLED 분기를 정확히 구현했다. A/B/C 버전에서 없었던 Cancel이 D에서 등장한 핵심 원인이다.

02

화면명세의 효과

D2에서 [화면 구성] 블록 추가만으로 task_history·notifications 2개 테이블과 NotificationService가 추가 생성되었다. 텍스트 수준의 화면 명세도 AI 추론에 유의미한 영향을 준다.

03

여전한 한계

AI 분석 엔진 시뮬레이션 방식, 예외 클래스 명명, 테스트 구조는 D1·D2에서도 다르게 추론되었다. Use Case Description 없이는 비즈니스 로직 흐름이 AI 임의 결정에 의존한다.

→

다음 회차 (WS1)

Use Case Modeling 추가 후 D 버전과 비교하여 Actor 분리, 대안 흐름 명세가 코드 구조에 어떤 변화를 가져오는지 관찰한다.